

USB Hardware Research — TEAS Study App

Executive Summary

- **Recommended drive:** SanDisk Ultra Fit 64GB (USB 3.1 Gen 1) — low profile so it stays plugged in during study sessions, fast enough for app delivery, fits budget
- **Target cost:** \$8-12/unit retail; \$6-9/unit bulk with branding at 50-unit volume
- **USB-A connector required** — broadest compatibility with student laptops; USB-C is gaining but adapters add cost and friction
- **Portable Python (embeddable) + PyInstaller** is the simplest path to a no-install Windows app on USB
- **Autorun is dead** on Windows 10/11 — students will double-click a `START.exe` or shortcut on the drive; include a printed quick-start card with each USB

USB Standards & Performance

Which USB Standard We Need

USB 3.2 Gen 1 (aka USB 3.0 / USB 3.1 Gen 1 / 5 Gbps) is the sweet spot. Here's why:

Standard	Speed	Flash drive availability	Verdict
USB 2.0	480 Mbps	Everywhere	Too slow for running apps — noticeable lag
USB 3.2 Gen 1	5 Gbps	Common, affordable	Minimum recommended
USB 3.2 Gen 2	10 Gbps	Rare in flash drives	Overkill, premium price
USB4 / USB 3.2 Gen 2×2	20-40 Gbps	Not available in thumb drives	Irrelevant

All USB 3.x standards are **backwards compatible** with USB 2.0 ports — the drive will work at USB 2.0 speed if plugged into an older port. This matters for community college thin clients that may have older hardware.

Random I/O vs Sequential Speed

Random I/O matters far more than sequential read speed for running an application:

- **Sequential speed** (the big MB/s number manufacturers advertise) = reading one large file continuously. Good for copying a video.
- **Random I/O** = reading hundreds of small files scattered across the drive. This is what running an app actually does — loading Python runtime, importing modules, reading config files, accessing a SQLite database.

Flash drives typically have **terrible random I/O** compared to SSDs. A USB 3.0 drive claiming "400 MB/s read" might only do 1–5 MB/s on random 4K reads. This means: - App startup will be slower than from a local SSD (acceptable trade-off) - Choose drives from reputable brands (Samsung, SanDisk) — they use better controllers and NAND that improve random I/O - **Keep the app lean** — minimize dependencies, bundle only what's needed, use a single-file executable where possible

USB-A vs USB-C

Factor	USB-A	USB-C
Student laptop compatibility	~95%+ of Windows laptops have USB-A	Growing but not universal, especially older models
Thin client compatibility	Almost certainly available	Unlikely on campus thin clients
Adapter cost	None	\$5–10 per USB-A→C adapter if needed
Recommendation	Use USB-A	Not for first deployment

Decision: USB-A only. Don't make students figure out adapters. If a student only has USB-C, include a small USB-A→C adapter in the package as a fallback option (adds ~\$1/unit).

Recommended USB Drives

Model	Capacity	Price (retail)	Read Speed	Write Speed	Pros	Cons
SanDisk Ultra Fit	32/64/128/256GB	\$8–18	Up to 400 MB/s	Up to 150 MB/s	Nano form factor — stays plugged in,	Low profile means hard to grip for

Model	Capacity	Price (retail)	Read Speed	Write Speed	Pros	Cons
					hard to snap off; widely available; cheap in bulk	removal; can get warm under sustained use
Samsung FIT Plus	32/64/128/256GB	\$10-20	Up to 400 MB/s	Up to 100 MB/s	Small form factor; Samsung NAND quality; 5-proof (water/shock/temp/magnet/X-ray); good controller	Slightly more expensive; write speeds lower than SanDisk
SanDisk Extreme Pro	32/64/128/256GB	\$12-22	Up to 420 MB/s	Up to 380 MB/s	Best write speeds; retractable connector (protected); good for frequent rewrites	Bulkiest of the three; most expensive; retractable mechanism can wear
Corsair Flash Voyager	32/64/128GB	\$15-25	Up to 200 MB/s	Up to 40 MB/s	Rubber housing — very durable; water resistant; good for rough handling	Slowest write speed; overpriced for specs; rubber attracts lint in pockets

Notes on Durability

For student use, durability matters. Students will toss these in backpacks, drop them, spill coffee. Key durability factors: - **NAND type:** 3D NAND (Samsung, modern SanDisk) has better endurance than planar NAND - **Housing:** Metal > plastic for impact resistance - **Connector:** Retractable or capped protects against bent pins - **The SanDisk Ultra Fit's nano design** is a double-edged sword — hard to break but easy to forget in a laptop

Portable App Technology

How to Make Our App Run from USB (No Install)

The goal: student plugs in USB, opens the drive in File Explorer, double-clicks one file, app launches. No installer, no admin rights, no dependencies on the host machine.

Recommended approach: PyInstaller single-file executable

1. Develop the app normally (Python + Flask for a browser-based UI, or a native GUI with tkinter/customtkinter)
2. Use PyInstaller to bundle into a single .exe file: `bash pyinstaller --onefile --windowed --name TEAS_Study app.py`
3. Place TEAS_Study.exe in the USB root alongside a data/ folder
4. Student double-clicks TEAS_Study.exe — it just works

This produces a single file (typically 30–80 MB) that contains Python interpreter + all dependencies. No installation needed.

Portable Python Approach

If PyInstaller's single-file approach is too slow to launch (it extracts to a temp dir each time), use **Python Embeddable Package**:

1. Download the official Windows embeddable Python zip from python.org
2. Extract to USB_DRIVE/python/
3. Install packages with pip into that directory
4. Create a START.bat launcher: `bat @echo off cd /d %~dp0 python\python.exe app.py pause`
5. Add required .pth files or site-packages for dependencies

WinPython is an alternative — it's a pre-built portable Python distribution with packages like NumPy, Pandas, etc. pre-installed. Useful if the app needs heavy scientific computing, but likely overkill for a study app.

Python Embeddable is preferred — it's minimal (~15 MB), official, and we control exactly what's included.

Windows Auto-Launch Without Autorun

Autorun.inf is disabled on Windows 10/11 for USB drives — Microsoft blocked this in 2011 (security: Stuxnet). There is NO way to auto-launch an executable from a USB drive on modern Windows.

Practical alternatives (ranked by user experience):

1. **START.bat in root** — student opens USB drive, double-clicks the .bat file. Name it something obvious: 👉 START TEAS STUDY APP.bat
2. **Desktop shortcut** — include a setup_shortcut.bat that copies a shortcut to the student's desktop. This is a one-time action and makes subsequent launches trivial.
3. **AutoPlay dialog** — Windows shows an AutoPlay notification when USB is inserted. By placing an autorun.inf with an open= entry, the notification may show a "Run program" option (Windows 10 still shows this in some cases, though Windows 11 has further restricted it). Not reliable — don't depend on it.
4. **Printed quick-start card** — include a small laminated card with each USB: "1. Plug in USB 2. Open 'TEAS Study' drive 3. Double-click START"

Recommended combo: START.bat + desktop shortcut installer + printed card.

Saving User Progress (Local Storage on USB)

Store all user data on the USB drive itself — this makes the USB self-contained and lets the student use any computer:

```
TEAS_Study/
├─ START.bat
├─ app/
│   └─ teas_study.exe          (or python/ + app.py)
├─ data/
│   ├── user_progress.db      (SQLite – scores, completed sections)
│   ├── notes/
│   └─ settings.json
└─ README.txt
```

- **SQLite** for structured data (progress, scores, bookmarks) — single file, no server needed, resilient to USB removal
- **JSON** for settings and preferences — simple to read/write
- **File-based** for any user notes or practice essays

Important: SQLite on USB flash drives is safe for reads but can be fragile on writes if the USB is removed mid-write. Add: - WAL (Write-Ahead Logging) mode for SQLite — more crash-resistant - Periodic auto-save (every 30–60 seconds) - A clean shutdown handler that flushes data on app exit - Backup: periodically copy user_progress.db to user_progress_backup.db

Bulk Pricing & Branding

Volume Pricing Estimates (32GB USB 3.0, custom branded)

Volume	Price per unit (est.)	Custom branding available?	Notes
10 units	\$10–15	Yes (some suppliers)	Retail pricing, minimal bulk discount
25 units	\$8–12	Yes	Entry-level bulk; many suppliers start here
50 units	\$6–10	Yes	Sweet spot for small program; most suppliers offer logo printing
100 units	\$5–8	Yes	Good pricing; can negotiate on data pre-loading
250 units	\$4–6	Yes	Best tier for community college program; room to grow
500+ units	\$3–5	Yes	Requires sourcing from China direct; 4–6 week lead time

Branding Options

Method	Cost (per unit)	Durability	Best for
Screen printing (1-color)	\$0.50–1.00	Good	Simple logo, high contrast
Full-color digital print + epoxy dome	\$1.00–2.00	Excellent	Complex logos, photo-quality
Laser engraving	\$0.75–1.50	Permanent (won't fade)	Metal drives only
Custom shape (mold)	\$2,000+ setup	N/A	Large runs (1000+), not practical for us

Recommended Suppliers

- **USB Memory Direct** (US-based) — good for 25–500 units, fast turnaround, pre-load data service
- **FlashBay** — global supplier, good quality control, 50-unit minimum, free digital proof

- **China direct (Alibaba/well-wholesale)** — cheapest (\$2–4/unit) but longer lead time (3–6 weeks), quality varies, USB 2.0 common (must specify 3.0)
- **SanDisk/Samsung direct** — possible for large orders, but custom branding is limited on retail products

Data Pre-Loading

Most bulk suppliers offer data pre-loading: they flash the app and all content onto every USB before shipping. Cost: typically \$0.25–0.50/unit. This saves hours of manual imaging.

Recommendation

Specific Recommendation

Drive: SanDisk Ultra Fit 64GB (USB 3.1 Gen 1, USB-A) **Capacity:** 64GB — provides headroom beyond the 32GB minimum for future content expansion **Technology stack:**
 - Python Embeddable Package (portable Python on USB) - PyInstaller single-file .exe (fallback if embeddable has issues) - SQLite with WAL mode for user progress - Flask browser-based UI (launches local server, opens default browser) — no GUI framework needed - START.bat launcher + optional desktop shortcut installer

Estimated total cost per unit (at 50 units): | Component | Cost | |-----|-----| |
 SanDisk Ultra Fit 64GB (bulk) | \$7–9 | | Custom logo printing (1-color) | \$0.75–1.00 | |
 Data pre-loading | \$0.25–0.50 | | Quick-start card (printed) | \$0.15–0.25 | | **Total per unit** | **\$8.15–10.75** |

This is **well under the \$15/unit budget** and leaves room for a USB-A→C adapter (\$1 each) for the few students who need one.

Why Not the Others

- **Samsung FIT Plus:** Good alternative if SanDisk supply is constrained. Nearly identical specs.
- **SanDisk Extreme Pro:** Overkill on write speed (we're mostly reading). Bulky form factor — students will leave it sticking out of their laptop.
- **Corsair Flash Voyager:** Too slow, too expensive. The rubber housing doesn't justify the price premium.

Risk Factors

1. **USB removal during write** — could corrupt SQLite. Mitigate with WAL mode + backup + user education ("don't remove USB while app is running")

2. **Thin client lockdown** — if campus computers block USB entirely (some IT departments do), students must use personal laptops. Investigate campus IT policy before ordering.
3. **Antivirus false positives** — PyInstaller executables sometimes trigger Windows Defender. Test on multiple machines; may need to sign the executable (\$\$\$) or whitelist.
4. **USB port speed variance** — some older thin clients only have USB 2.0. App should still work, just slower. Test on USB 2.0 during development.